

1.What is multithreading in Java?

Multithreading in Java is a programming feature that allows **multiple threads (lightweight units of a process) to run concurrently** within a single program. Each thread can execute a separate task independently, sharing the same memory space, which improves CPU utilization and application performance.

In simple terms: Java lets your program do **many things at the same time** by running multiple threads in parallel.

For example:

- Playing music while downloading a file
- Handling multiple user requests in a web server

2. What are the benefits of multithreading?

The **benefits of multithreading in Java** are:

1. **Better CPU Utilization** – Threads run concurrently, keeping the CPU busy and improving performance.
2. **Faster Execution** – Parallel tasks complete more quickly compared to sequential execution.
3. **Improved Responsiveness** – Applications remain responsive (e.g., GUI apps don't freeze while performing background tasks).
4. **Efficient Resource Sharing** – Threads share memory and resources of the process, reducing overhead.
5. **Simpler Program Structure for Concurrent Tasks** – Easier to manage multiple simultaneous operations like I/O, networking, or calculations.

3.What is the difference between a process and a thread?

Feature	Process	Thread
Definition	Independent program in execution	Lightweight subunit of a process
Memory	Has its own separate memory space	Shares memory of the parent process
Communication	Uses inter-process communication (IPC)	Can communicate directly via shared memory
Creation Overhead	High	Low
Execution	Runs independently	Runs as part of a process
Use Case	Running different applications	Running concurrent tasks within an application

💡 **In short:**

- **Process** = heavy, independent program

- **Thread** = lightweight, shares resources, runs inside a process

4. Explain the lifecycle of a thread in Java.?

The **lifecycle of a thread in Java** describes the different states a thread goes through from creation to termination.

explanation with states:

1. New (Created)

- Thread object is created using Thread class or Runnable interface.
- Thread is not yet started.

```
Thread t = new Thread();
```

2. Runnable (Ready to run)

- After calling start(), the thread is **ready to run**.
- It waits for the **CPU scheduler** to allocate CPU time.

```
t.start();
```

3. Running

- The thread is **executing its run() method**.
- Only one thread executes at a time per CPU core.

4. Waiting / Blocked

- Thread waits **indefinitely** until another thread signals it.
- Example: wait() method or waiting for a resource.

5. Timed Waiting

- Thread waits for a **specified time** before resuming.
- Example: sleep(milliseconds) or join(milliseconds)

6. Terminated (Dead)

- Thread has **completed execution** or stopped.
- Cannot be restarted.

summary diagram:

New → Runnable → Running → Waiting / Timed Waiting → Running → Terminated

6.What are the different states of a thread?

In Java, a thread can exist in **six main states** as defined by the Thread.State enum:

1. **New** – Thread is created but **not yet started**.

```
Thread t = new Thread();
```

2. **Runnable** – Thread is **ready to run** and waiting for CPU scheduling.
3. **Running** – Thread is **executing its run() method**.
4. **Waiting** – Thread is **indefinitely waiting** for another thread to notify it.
 - Example: wait() without timeout
5. **Timed Waiting** – Thread is waiting for a **specific time period**.
 - Example: sleep(milliseconds), join(milliseconds)
6. **Terminated (Dead)** – Thread has **completed execution** and cannot be restarted.

Summary diagram:

New → Runnable → Running → Waiting / Timed Waiting → Running → Terminated

7. What is the difference between Thread class and Runnable interface?

Feature	Thread Class	Runnable Interface
Inheritance	Extends Thread class	Implements Runnable interface
Multiple Inheritance	Not possible (Java doesn't support multiple class inheritance)	Possible (can extend another class while implementing Runnable)
Usage	Directly create and start a thread	Create a thread by passing Runnable object to Thread
Flexibility	Less flexible	More flexible
Shared Resource	Each thread is separate object	Multiple threads can share the same Runnable object

Example:

Using Thread class:

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread running");
    }
}
MyThread t = new MyThread();
t.start();
```

Using Runnable interface:

```
class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Thread running");
    }
}
```

```
Thread t = new Thread(new MyRunnable());
t.start();
```

NOTE:

- Use **Runnable** when you want to **implement multithreading while still extending another class**.
- Use **Thread class** for **simple cases** when no other inheritance is needed.

8. How do you create a thread in Java?

In Java, you can create a thread in two main ways:

1. By Extending the Thread Class

- Create a subclass of Thread and override its run() method.
- Call start() to begin execution.

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread is running");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        t1.start(); // Starts a new thread
    }
}
```

2. By Implementing the Runnable Interface

- Implement Runnable and override run().
- Pass the Runnable object to a Thread and call start().

```
class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Thread is running");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Thread t1 = new Thread(new MyRunnable());
        t1.start(); // Starts a new thread
    }
}
```

Summary:

- Use Runnable when you need to extend another class (since Java allows only single inheritance).
- Use Thread class for simple, standalone threads.

9. What is the difference between start() and run() methods?

Feature	start()	run()
Purpose	Creates a new thread and calls run() internally	Executes run() in the current thread, no new thread created
Thread Creation	Yes, a new thread is created	No, just a normal method call
Concurrency	Enables concurrent execution	Executes sequentially in current thread
Typical Usage	Always call start() to run thread	Rarely called directly, only for testing

Example:

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread running: " + Thread.currentThread().getName());
    }
}

public class Main {
    public static void main(String[] args) {
        MyThread t = new MyThread();
        t.start(); // Runs in a new thread
        // t.run(); // Runs in main thread (no new thread)
    }
}
```

Summary:

- Always use start() to actually run threads concurrently.
- Calling run() directly just executes the method like a normal function.

10. What is the difference between sleep() and wait() in Java?

Feature	sleep()	wait()
Class	Thread class	Object class
Lock Release	Does NOT release the lock	Releases the lock
Purpose	Pauses thread for a specific time	Makes thread wait until notified

Feature	sleep()	wait()
Usage Context	Can be called anywhere	Must be called inside synchronized block/method
Resume	Automatically resumes after time expires	Resumes only when another thread calls notify() or notify All()

Example:

Using sleep()

```
Thread.sleep(1000); // pauses for 1 second
```

Using wait()

```
synchronized(obj) {
    obj.wait(); // waits until notify() is called
}
```

Summary:

- **sleep()** → simple pause
- **wait()** → coordination between threads using monitor/lock

11. What is the difference between yield() and sleep()?

Feature	yield()	sleep()
Purpose	Suggests scheduler to give CPU to other threads of same priority	Pauses the thread for a specific time
Thread State	Thread goes from Running → Runnable (ready to run again)	Thread goes to Timed Waiting state
Lock Release	Does not release locks	Does not release locks
Duration	No fixed time, may run immediately again	Fixed time specified in milliseconds
Use Case	Let other threads of same priority run	Pause execution, e.g., for delay or simulation

Example:

```
// yield()
Thread.yield();
```

```
// sleep()
Thread.sleep(1000); // pauses for 1 second
```

Memory Tip:

- **yield()** = “I’m polite, let others run”

- `sleep()` = “I’m taking a nap for X ms”

Synchronization & Thread Safety

11. What is synchronization in Java?

In Java, synchronization is a mechanism that controls the access of multiple threads to shared resources to prevent data inconsistency and race conditions. When multiple threads try to access or modify a shared resource simultaneously, synchronization ensures that only one thread can access the resource at a time.

Key Points:

1. Purpose:
 - Prevent race conditions, where two or more threads access shared data concurrently, leading to incorrect results.
 - Ensure thread safety when multiple threads work on shared data.
2. How it works:
 - Java uses a monitor (or intrinsic lock) for every object.
 - When a thread enters a synchronized method/block, it acquires the lock of the object.
 - Other threads trying to access the same synchronized block/method have to wait until the lock is released.
3. Ways to synchronize in Java:

a) Synchronized Method

```
public synchronized void increment() {  
    count++;  
}
```

- The entire method is synchronized.
- Only one thread can execute it at a time for a particular object.

b) Synchronized Block

```
public void increment() {  
    synchronized(this) {  
        count++;  
    }  
}
```

- Only the critical section is synchronized, improving performance.

4. Static Synchronization:

- If a method is static synchronized, the class-level lock is used, not the object-level lock.

5. Benefits:

- Ensures data consistency.
- Makes methods thread-safe.

6. Drawbacks:

- Can cause performance overhead due to waiting threads.
- May lead to deadlocks if not used carefully.

Example: Race Condition Without Synchronization

```
class Counter {
    int count = 0;
    void increment() {
        count++;
    }
}

public class Test {
    public static void main(String[] args) throws InterruptedException {
        Counter c = new Counter();
        Runnable r = () -> {
            for(int i=0; i<1000; i++) c.increment();
        };
        Thread t1 = new Thread(r);
        Thread t2 = new Thread(r);
        t1.start();
        t2.start();
        t1.join();
        t2.join();
        System.out.println(c.count); // May not be 2000 without synchronization
    }
}
```

With Synchronization:

```
synchronized void increment() {
    count++;
}
```

- Now, count will always be 2000, because only one thread increments at a time

12. How do you make a method thread-safe?

In **Java**, making a method **thread-safe** means ensuring that it behaves correctly when accessed by **multiple threads simultaneously**, without causing **race conditions or inconsistent data**. There are several ways to do this.

1. Using synchronized Keyword

a) Synchronized Method


```
public synchronized void increment() {
    count++;
}
```

- The **entire method** is locked on the object.
- Only **one thread** can execute it at a time per instance.
- For **static methods**, the lock is at the **class level**.

b) Synchronized Block

```
public void increment() {
    synchronized(this) {
        count++;
    }
}
```

- Locks **only the critical section** instead of the whole method.
- Better performance than synchronizing the whole method.

2. Using ReentrantLock

- Provides more **flexibility** than synchronized.

```
import java.util.concurrent.locks.ReentrantLock;
```

```
class Counter {
    private int count = 0;
    private ReentrantLock lock = new ReentrantLock();

    public void increment() {
        lock.lock();
        try {
            count++;
        } finally {
            lock.unlock();
        }
    }
}
```

- Can **try to acquire lock, interruptible lock, or fair locking**.

3. Using Atomic Classes

- Java provides classes in java.util.concurrent.atomic package for **lock-free thread safety**.

```
import java.util.concurrent.atomic.AtomicInteger;
```

```
class Counter {
    private AtomicInteger count = new AtomicInteger(0);

    public void increment() {
```

```

        count.incrementAndGet(); // Thread-safe
    }
}

```

- No explicit locking needed.
- **High-performance** for simple operations like increment.

4. Making Method Immutable

- If the method **does not modify shared state** or uses **local variables only**, it's inherently **thread-safe**.

```

public int add(int a, int b) {
    return a + b; // no shared state
}

```

Summary Table

Approach	When to Use	Notes
synchronized method	Simple, small-scale shared resources	Can reduce performance if method is long
synchronized block	Lock only critical section	Better performance than method-level locking
ReentrantLock	Advanced locking features	Can use tryLock, fairness, interruptible lock
AtomicInteger / Atomic classes	Simple numeric/shared operations	Lock-free, high performance
Immutable / stateless method	When no shared state is accessed	Naturally thread-safe

13. What is a synchronized block?

In Java, a synchronized block is a way to synchronize only a specific section of code, rather than the entire method. This helps improve performance by reducing the time a thread holds a lock.

Key Points

1. Purpose:
 - To control access to a shared resource among multiple threads.
 - Only one thread can execute the synchronized block for a given object lock at a time.
2. Syntax:

```

synchronized(lockObject) {
    // Critical section (shared resource access)
}

```

- lockObject is the object whose lock is being used.
- Threads must acquire the lock on lockObject before entering the block.
- Once the block is finished, the lock is released.

Example

```
class Counter {
    private int count = 0;

    public void increment() {
        synchronized(this) { // 'this' object lock
            count++;
        }
    }

    public void decrement() {
        synchronized(this) { // same lock object
            count--;
        }
    }
}
```

- Only one thread can execute increment() or decrement() at a time.
- Threads trying to enter while the lock is held must wait.

Why Use Synchronized Block Instead of Method?

Synchronized Method	Synchronized Block
Locks entire method	Locks only critical section
Simple to implement	More control, better performance
May reduce performance if method is long	Efficient, only locks what's necessary

Example with Specific Object

```
class Counter {
    private int count = 0;
    private Object lock = new Object(); // custom lock

    public void increment() {
        synchronized(lock) {
            count++;
        }
    }
}
```

```
}  
}
```

- Using a custom lock object allows fine-grained synchronization.
- Other methods can use different locks, avoiding unnecessary blocking.

Summary:

- A synchronized block allows thread-safe access to shared resources without locking the whole method.
- You can choose which object's lock to use, making it more flexible and efficient than synchronized methods.

14. What is a reentrant lock?

A **ReentrantLock** is a class in `java.util.concurrent.locks` that provides **explicit locking** with more flexibility than the `synchronized` keyword.

It is **reentrant**, meaning a thread that already holds the lock can **acquire it again without blocking**, maintaining a **hold count** internally.

Key points:

1. **Thread-safe:** Ensures that only one thread executes the critical section at a time.
2. **Advanced features over synchronized:**
 - Can attempt to acquire the lock without waiting using `tryLock()`.
 - Supports **interruptible lock acquisition** with `lockInterruptibly()`.
 - Can implement **fairness policy** to grant locks in request order.
3. **Usage:** Always pair `lock()` with `unlock()` in a try-finally block to avoid deadlocks.

Example:

```
ReentrantLock lock = new ReentrantLock();
```

```
lock.lock();  
try {  
    // critical section  
} finally {  
    lock.unlock();  
}
```

15. Difference between synchronized and Lock interface.?

Both `synchronized` and `Lock` (like `ReentrantLock`) are used for thread synchronization, but they have key differences:

Feature	synchronized	Lock (ReentrantLock)
Declaration	Keyword	Interface (java.util.concurrent.locks.Lock)
Flexibility	Less flexible; entire method or block is locked	Highly flexible; can lock/unlock parts of code, try to acquire lock, interruptible locks
Lock Release	Automatically released when block/method exits	Must manually call unlock() in finally block
Reentrancy	Yes, thread can re-enter the same lock	Yes, supports reentrant locking
Fairness	No option	Can create fair lock to grant access in request order
Interruptible Lock Acquisition	No	lockInterruptibly() allows a waiting thread to be interrupted
Performance	Simpler, less overhead for basic cases	Slightly higher overhead but better control in complex scenarios

Example Comparison:

Synchronized

```
public synchronized void increment() {
    count++;
}
```

Lock Interface

```
ReentrantLock lock = new ReentrantLock();
lock.lock();
try {
    count++;
} finally {
    lock.unlock();
}
```

Summary:

- Use synchronized for simple synchronization.
- Use Lock interface when you need advanced features, fairness, interruptibility, or fine-grained control.

16. What is deadlock? How do you prevent it?

Deadlock is a situation in multithreading where two or more threads are blocked forever, each waiting for a resource held by another thread.

- In other words, threads are waiting on each other and none can proceed.
- It usually happens when multiple threads acquire multiple locks in different orders.

Example

Thread 1: locks A, then tries to lock B

Thread 2: locks B, then tries to lock A

- Both threads wait indefinitely → deadlock.

Conditions for Deadlock (Coffman Conditions)

1. Mutual Exclusion – A resource can be held by only one thread at a time.
2. Hold and Wait – A thread holding one resource waits for another.
3. No Preemption – Resources cannot be forcibly taken from threads.
4. Circular Wait – A circular chain of threads exists, each waiting for a resource held by the next.

How to Prevent Deadlock

1. Avoid Nested Locks / Reduce Lock Scope
 - Acquire locks only when needed and release quickly.
2. Lock Ordering
 - Always acquire multiple locks in a consistent global order.
3. Try-Lock with Timeout (ReentrantLock)

```
if(lock.tryLock(1000, TimeUnit.MILLISECONDS)) {  
    try {  
        // critical section  
    } finally {  
        lock.unlock();  
    }  
} else {  
    // handle lock acquisition failure  
}
```

4. Avoid Circular Wait
 - Ensure that the application design does not allow a circular chain of waiting threads.
5. Use Higher-Level Concurrency Utilities
 - Use classes from `java.util.concurrent` (like Semaphore, ConcurrentHashMap) that handle internal locking safely.

17. What is livelock and starvation in multithreading?

1. Livelock

Definition:

- Livelock occurs when **threads are not blocked** but **keep changing their states** in response to each other, so **no thread can make progress**.
- Unlike deadlock, **threads are actively running** but still cannot complete their task.

Example:

- Two threads trying to politely give up a resource to each other continuously:

Thread 1 sees Thread 2 is using resource → gives way

Thread 2 sees Thread 1 is giving way → gives way

...repeats forever

Threads are **alive**, but **no useful work is done**.

2. Starvation

Definition:

- Starvation occurs when a thread **waits indefinitely** because higher-priority threads **continuously preempt resources**, so it **never gets CPU time**.

Example:

- Thread priority-based scheduling:

High-priority threads keep executing → low-priority thread never runs

Starvation is about **unfair access**, some threads **may never execute**.

Difference between Livelock and Starvation

Feature	Livelock	Starvation
Thread state	Active (running)	Waiting indefinitely
Cause	Threads repeatedly respond to each other	Threads never get CPU or resource due to priority or unfair scheduling
Work done	No progress despite activity	No progress because thread is never scheduled
Example	Polite threads repeatedly yielding	Low-priority thread never runs because high-priority threads dominate

How to Prevent

Livelock:

- Use proper **coordination** between threads.
- Introduce **random back-off** or **timeouts** to break the cycle.

Starvation:

- Use **fair locks** (`ReentrantLock(true)`), which give threads access in **request order**.
- Avoid **thread priority-based unfair scheduling**.

18. What is the difference between process synchronization and thread synchronization?

Feature	Process Synchronization	Thread Synchronization
Definition	Coordination between two or more processes to access shared resources safely	Coordination between threads within the same process to access shared resources safely
Memory Sharing	Processes do not share memory by default; need IPC (Inter-Process Communication) like pipes, shared memory, sockets	Threads share the same memory within a process, so synchronization is easier
Mechanism	Uses semaphores, message passing, shared memory	Uses synchronized keyword, locks (ReentrantLock), atomic classes, volatile variables
Overhead	Higher overhead due to context switching and IPC mechanisms	Lower overhead because threads share the same address space
Example	Two programs updating a file simultaneously	Two threads updating a shared <code>HashMap</code> or <code>Counter</code> in memory

19. What is volatile keyword? When should you use it?

Definition:

- In Java, the **volatile keyword** is used to **mark a variable so that all threads see its latest value immediately**.
- It guarantees **visibility** of changes across threads, but **does not provide atomicity**.

Key Points

1. Visibility Guarantee:

- When a thread updates a volatile variable, the new value is **written directly to main memory**, not just the CPU cache.
- Other threads reading the variable **see the updated value immediately**.

2. No Atomicity:

- Operations like `count++` are **not atomic**, even if the variable is volatile.
- For atomic operations, use **`AtomicInteger`** or **`synchronized blocks`**.

3. When to Use volatile:

- When a variable is **shared between threads** and **read/written independently**.

- Ideal for **flags, state indicators, or stop signals** in multithreaded programs.

Example

```
class StopThreadExample {
    private volatile boolean stop = false;

    public void run() {
        while(!stop) {
            // do some work
        }
        System.out.println("Thread stopped");
    }

    public void requestStop() {
        stop = true; // visible to other threads immediately
    }
}
```

- Without volatile, the thread **may not see the updated value** of stop and could loop forever.

Do Not Use volatile When:

- Performing **compound operations** (like increment/decrement): count++
- Multiple variables need to be updated **atomically together** → use **synchronized blocks** or **atomic classes**.

20. Difference between Atomic classes and synchronized keyword.?

Atomic classes (like AtomicInteger, AtomicLong) and the synchronized keyword are both used to achieve thread safety, but they work differently:

Feature	synchronized	Atomic Classes
Mechanism	Uses intrinsic locks to make a method or block mutually exclusive	Uses low-level CPU atomic instructions (like Compare-And-Swap) for specific operations
Performance	Slower due to locking overhead	Faster for simple operations (increment, decrement, update) because it's lock-free
Scope	Can make any code block or method thread-safe	Only provides atomic operations on single variables
Flexibility	Can synchronize multiple statements or objects	Only supports predefined atomic operations on the variable
Deadlock Risk	Possible if not used carefully	No deadlock risk because it's lock-free

Feature	synchronized	Atomic Classes
Usage	synchronized void	AtomicInteger count = new AtomicInteger();
Example	increment() { count++; }	count.incrementAndGet();

Summary:

1. Use synchronized when you need complex operations or multiple statements to be thread-safe.
2. Use Atomic classes for simple atomic operations on single variables with better performance.
3. Atomic classes are lock-free, so they avoid thread blocking and deadlocks.

Advanced Java Multithreading

21. What is the Executor framework in Java?

The Executor Framework in Java (introduced in Java 5) is a high-level API for managing and controlling threads.

It decouples task submission from thread management, allowing developers to focus on the tasks rather than creating and managing threads manually.

1. Purpose:
 - Simplifies thread management.
 - Avoids creating new threads manually (new Thread()), which is inefficient for large applications.
2. Core Interfaces:
 - Executor – Basic interface for executing tasks.
 - ExecutorService – Extends Executor; provides lifecycle management (shutdown, termination).
 - ScheduledExecutorService – Supports delayed and periodic tasks.
3. Common Implementations:
 - ThreadPoolExecutor – Most flexible; manages a pool of threads.
 - ScheduledThreadPoolExecutor – Executes tasks after a delay or periodically.

4. Usage Example:

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class ExecutorExample {
    public static void main(String[] args) {
        ExecutorService executor = Executors.newFixedThreadPool(3); // Thread pool of 3 threads

        for (int i = 1; i <= 5; i++) {
```

```

        int taskId = i;
        executor.submit(() -> {
            System.out.println("Executing task " + taskId + " by " +
                Thread.currentThread().getName());
        });
    }

    executor.shutdown(); // Initiates orderly shutdown
}

```

- Tasks are queued and executed by available threads in the pool.
- Reduces thread creation overhead and improves performance.

Advantages

1. Better resource management – Reuses threads instead of creating new ones.
2. Scalable – Handles large numbers of concurrent tasks efficiently.
3. Simpler code – No need to manage thread lifecycle manually.
4. Supports scheduling – Delayed or periodic task execution.

22. What is the difference between Executor, ExecutorService, and ScheduledExecutorService?

In Java, all three are part of the **Executor Framework** for managing threads, but they serve slightly different purposes:

Interface	Description	Key Features
Executor	Basic interface for executing tasks	- Single method: <code>execute(Runnable command)</code> - Decouples task submission from thread creation - No lifecycle management (cannot shutdown)
ExecutorService	Extends Executor; provides thread pool management	- Methods: <code>submit()</code>, <code>invokeAll()</code>, <code>shutdown()</code>, <code>awaitTermination()</code> - Supports returning results via Future - Manages thread lifecycle
ScheduledExecutorService	Extends ExecutorService; supports scheduled and periodic tasks	- Methods: <code>schedule()</code>, <code>scheduleAtFixedRate()</code>, <code>scheduleWithFixedDelay()</code> - Execute tasks after a delay or periodically

Key Points for Interviews:

1. **Executor** → simplest, just executes tasks.
 2. **ExecutorService** → adds **thread pooling, lifecycle management, and result handling**.
 3. **ScheduledExecutorService** → adds **timed and periodic execution** on top of ExecutorService.
-

Example:

```
Executor executor = Executors.newSingleThreadExecutor();
executor.execute(() -> System.out.println("Task using Executor"));
```

```
ExecutorService executorService = Executors.newFixedThreadPool(2);
Future<Integer> result = executorService.submit(() -> 10 + 20);
```

```
ScheduledExecutorService scheduledExecutor = Executors.newScheduledThreadPool(1);
scheduledExecutor.schedule(() -> System.out.println("Delayed Task"), 5, TimeUnit.SECONDS);
```

23. Explain ThreadPoolExecutor in Java.?

ThreadPoolExecutor is a flexible and highly configurable implementation of the ExecutorService interface in Java.

It manages a pool of threads to execute tasks concurrently, which reduces the overhead of creating new threads for every task.

Key Points

1. Core Concept:
 - Instead of creating a new thread for each task, ThreadPoolExecutor reuses threads from a thread pool.
 - Tasks are queued if no thread is available, and executed when a thread becomes free.
2. Core Parameters:

Parameter	Description
corePoolSize	Minimum number of threads kept alive in the pool, even if idle
maximumPoolSize	Maximum number of threads allowed in the pool
keepAliveTime	Time that extra threads (beyond corePoolSize) wait before terminating
workQueue	Queue to hold tasks before they are executed (e.g., LinkedBlockingQueue)
threadFactory	Factory to create new threads (customizable)
RejectedExecutionHandler	Policy for handling tasks when the pool is full
3. Task Submission:
 - `execute(Runnable task)` – Fire-and-forget execution.

- submit(Callable/Void) – Can return a Future for result handling.

Example

```
import java.util.concurrent.*;

public class ThreadPoolExecutorExample {
    public static void main(String[] args) {
        ThreadPoolExecutor executor = new ThreadPoolExecutor(
            2, 5, 60, TimeUnit.SECONDS, new LinkedBlockingQueue<>()
        );

        for (int i = 1; i <= 10; i++) {
            int taskId = i;
            executor.execute(() -> {
                System.out.println("Executing Task " + taskId + " by " +
                    Thread.currentThread().getName());
            });
        }

        executor.shutdown();
    }
}
```

- Explanation:
 - Core threads = 2, max threads = 5.
 - Extra tasks wait in the work queue until threads are free.
 - Threads beyond corePoolSize are terminated after keepAliveTime if idle.

Advantages

1. Resource efficiency – Reuses threads instead of creating new ones.
2. High concurrency – Can handle a large number of tasks.
3. Flexible configuration – Control pool size, queue, and rejection policy.
4. Customizable behavior – Through threadFactory and RejectedExecutionHandler.

24. Difference between fixed thread pool and cached thread pool.?

In Java, both FixedThreadPool and CachedThreadPool are factory methods from Executors that create thread pools, but they behave differently:

Feature	Fixed Thread Pool	Cached Thread Pool
Thread Count	Fixed number of threads (nThreads)	Dynamically creates threads as needed, can grow unbounded
Reuse of Threads	Reuses a fixed set of threads	Reuses previously created threads if available; otherwise creates new ones
Idle Thread Handling	Threads are always alive	Threads that are idle for 60 seconds are terminated
Task Handling	Excess tasks wait in an unbounded queue	Excess tasks trigger creation of new threads immediately
Use Case	When the number of concurrent tasks is known and limited	When there are many short-lived tasks or bursty load
Example Creation	<code>Executors.newFixedThreadPool(5)</code>	<code>Executors.newCachedThreadPool()</code>

1. `FixedThreadPool` → predictable number of threads, good for consistent load.
2. `CachedThreadPool` → flexible, grows threads on demand, ideal for many short-lived tasks.
3. Resource Management: Fixed thread pool prevents resource exhaustion, while cached thread pool can grow threads aggressively if load spikes.

Example Usage:

```
ExecutorService fixedPool = Executors.newFixedThreadPool(3); // fixed 3 threads
ExecutorService cachedPool = Executors.newCachedThreadPool(); // dynamic threads
```

```
fixedPool.submit(() -> System.out.println("Task in fixed pool"));
cachedPool.submit(() -> System.out.println("Task in cached pool"));
```

25. What is a Callable and Future in Java?

1. Callable Interface:

- **Callable** is a functional interface in `java.util.concurrent` introduced in **Java 5**.
- Similar to `Runnable`, but it **can return a result** and **throw checked exceptions**.
- Signature:

`V call()` throws `Exception`;

2. Future Interface:

- **Future** represents the **result of an asynchronous computation**.
- It provides methods to **check if the task is done, wait for result, and cancel execution**.

Key Differences:

Feature	Runnable	Callable
Return Value	No	Yes (call() returns a value)
Exceptions	Cannot throw checked exceptions	Can throw checked exceptions
Used With	Thread, Executor	ExecutorService + Future

Feature Future

Purpose Represents **result of async computation**

get() – waits and retrieves result

Methods isDone() – check completion

cancel() – cancel task

Example:

```
import java.util.concurrent.*;

public class CallableFutureExample {
    public static void main(String[] args) throws Exception {
        ExecutorService executor = Executors.newFixedThreadPool(2);

        Callable<Integer> task = () -> {
            Thread.sleep(2000);
            return 42;
        };

        Future<Integer> future = executor.submit(task);

        System.out.println("Task submitted...");

        // Retrieve result (blocks until done)
        Integer result = future.get();
        System.out.println("Result: " + result);

        executor.shutdown();
    }
}
```

Output:

Task submitted...
Result: 42

1. **Callable** allows **returning a result and throwing exceptions**.
2. **Future** is used to **track and retrieve the result** asynchronously.
3. Can be combined with **ExecutorService** to execute tasks concurrently and get results.

26. How is Runnable different from Callable?

Runnable:

Runnable is a functional interface in Java (introduced in Java 1.0) that represents a task to be executed by a thread.

It cannot return a result and cannot throw checked exceptions.

The single method is:

```
void run();
```

Callable:

Callable is a functional interface in Java (introduced in Java 5, in `java.util.concurrent`) that represents a task that returns a result and can throw checked exceptions.

The single method is:

```
V call() throws Exception;
```

Differences

Feature	Runnable	Callable
Return Type	void	Generic V (any type)
Exceptions	Cannot throw checked exceptions	Can throw checked exceptions
Introduced in	Java 1.0	Java 5
Method Signature	void run()	V call() throws Exception
Execution	Thread or Executor	ExecutorService.submit() → returns Future<V>
Result Handling	None	Asynchronous via Future
Use Case	Fire-and-forget tasks	Tasks that require a result or exception handling

Example

Runnable Example:

```
Runnable runnableTask = () -> System.out.println("Runnable Task executed");  
new Thread(runnableTask).start();
```

Callable Example:


```
Callable<Integer> callableTask = () -> 42;
```

```
ExecutorService executor = Executors.newFixedThreadPool(1);  
Future<Integer> future = executor.submit(callableTask);
```

```
System.out.println("Callable result: " + future.get());  
executor.shutdown();
```

Key Points for Interviews

1. Runnable → simple, does not return a value.
2. Callable → returns a value, can throw exceptions, always used with ExecutorService.
3. Callable + Future allows asynchronous result retrieval and exception handling.

27.What is ForkJoinPool and ForkJoinTask?

Definition

ForkJoinPool:

ForkJoinPool is a specialized ExecutorService introduced in Java 7 to efficiently execute tasks that can be broken into smaller subtasks.

It implements the work-stealing algorithm, where idle threads “steal” tasks from busy threads to maximize CPU utilization.

ForkJoinTask:

ForkJoinTask is an abstract class representing a task that can be executed within a ForkJoinPool. Its two main subclasses are:

- RecursiveTask<V> → returns a result
- RecursiveAction → does not return a result

How It Works

1. Divide and Conquer: Large tasks are recursively split into smaller subtasks.
2. Fork: Subtasks are submitted to the pool using fork().
3. Join: Results of subtasks are combined using join().

Example – RecursiveTask

```
import java.util.concurrent.*;
```

```
class SumTask extends RecursiveTask<Integer> {  
    private final int[] arr;  
    private final int start, end;  
    private static final int THRESHOLD = 5;
```

```
    SumTask(int[] arr, int start, int end) {  
        this.arr = arr;  
        this.start = start;
```

```

        this.end = end;
    }

    @Override
    protected Integer compute() {
        if (end - start <= THRESHOLD) {
            int sum = 0;
            for (int i = start; i < end; i++) sum += arr[i];
            return sum;
        } else {
            int mid = (start + end) / 2;
            SumTask left = new SumTask(arr, start, mid);
            SumTask right = new SumTask(arr, mid, end);
            left.fork(); // execute asynchronously
            int rightResult = right.compute();
            int leftResult = left.join(); // wait and get result
            return leftResult + rightResult;
        }
    }
}

public class ForkJoinExample {
    public static void main(String[] args) {
        ForkJoinPool pool = new ForkJoinPool();
        int[] numbers = {1,2,3,4,5,6,7,8,9,10};
        SumTask task = new SumTask(numbers, 0, numbers.length);
        int result = pool.invoke(task);
        System.out.println("Sum: " + result);
        pool.shutdown();
    }
}

```

Output:

Sum: 55

1. ForkJoinPool → best for CPU-intensive, recursive, divide-and-conquer tasks.
2. Uses work-stealing to improve thread utilization.
3. ForkJoinTask → represents a task inside the pool; use RecursiveTask (returns value) or RecursiveAction (no value).
4. Avoid using ForkJoinPool for IO-bound tasks; it's optimized for CPU-bound tasks.

28. What is the difference between submit() and execute() in ExecutorService?

execute()

execute() is a method of ExecutorService used to submit a task for execution. It does not return any result and is suitable for fire-and-forget tasks. Exceptions thrown by the task are handled by the thread's uncaught exception handler.

submit()

submit() is a method of ExecutorService used to submit a task for execution and obtain a result asynchronously. It returns a Future object, which can be used to retrieve the result or handle exceptions thrown by the task. It supports both Runnable and Callable tasks.

Key Difference (One Liner)

- execute() → void, no result, fire-and-forget.
- submit() → Future, can get result, handles exceptions asynchronously.

In Java, both submit() and execute() are used to submit tasks to an **ExecutorService**, but they differ in **return type and behavior**.

Feature	execute()	submit()
Return Type	void	Future<V>
Can Return Result?	No	Yes, via Future
Exception Handling	Exceptions are handled by the thread's uncaught exception handler	Exceptions are captured in the Future and can be retrieved using future.get()
Use Case	Fire-and-forget tasks	Tasks where result or exception handling is needed
Task Types Accepted	Only Runnable	Runnable or Callable

Example

Using execute():

```
ExecutorService executor = Executors.newFixedThreadPool(2);
```

```
executor.execute(() -> {  
    System.out.println("Task executed using execute()");  
    // exceptions will be thrown to thread handler  
});
```

```
executor.shutdown();
```

Using submit():

```
ExecutorService executor = Executors.newFixedThreadPool(2);
```

```
Future<Integer> future = executor.submit(() -> {  
    return 10 + 20;  
});
```

```

try {
    System.out.println("Result from submit(): " + future.get());
} catch (Exception e) {
    e.printStackTrace();
}

executor.shutdown();

```

Key Points for Interviews

1. **execute()** → fire-and-forget, no result, only for Runnable.
2. **submit()** → returns Future, can handle **results and exceptions**, supports Runnable and Callable.
3. Use submit() when you **need a result or exception handling**, execute() when **you don't care about the result**.

29. Explain CompletableFuture in Java 8.?

CompletableFuture is a class in Java 8 (java.util.concurrent) that represents a future result of an asynchronous computation.

It extends Future and CompletionStage, allowing non-blocking, event-driven programming and chaining multiple asynchronous tasks.

Features:

1. Asynchronous Computation: Execute tasks without blocking the main thread.
2. Chaining: Supports thenApply(), thenAccept(), thenCompose(), etc., to chain dependent tasks.
3. Combining Futures: Can combine multiple futures using allOf() or anyOf().
4. Exception Handling: Supports exceptionally() and handle() for handling errors.
5. Non-blocking: Unlike Future.get(), it allows callbacks when the result is ready.

Example

```

import java.util.concurrent.CompletableFuture;

public class CompletableFutureExample {
    public static void main(String[] args) {
        // Asynchronous task
        CompletableFuture<Integer> future = CompletableFuture.supplyAsync(() -> {
            System.out.println("Running task in: " + Thread.currentThread().getName());
            return 10 + 20;
        });

        // Chaining another task
    }
}

```

```

CompletableFuture<Integer> result = future.thenApply(sum -> sum * 2);

// Retrieve result
result.thenAccept(finalResult -> System.out.println("Final Result: " + finalResult));

// Keep main thread alive to see output
try { Thread.sleep(1000); } catch (InterruptedException e) { e.printStackTrace(); }
}
}

```

Output:

Running task in: ForkJoinPool.commonPool-worker-1
Final Result: 60

1. Improved Future: Unlike Future, non-blocking and allows callbacks.
2. Functional Style: Works well with lambdas and streams.
3. Chaining & Composition: Easily combine or chain multiple asynchronous tasks.
4. Exception Handling: Handle exceptions gracefully using exceptionally() or handle().
5. Common Use Cases: Asynchronous APIs, parallel processing, microservices calls, and event-driven programming.

30. What are daemon threads?

Daemon threads in Java are background threads that run in the background to perform supporting tasks.

They are low-priority threads that do not prevent the JVM from exiting when all user threads have finished execution.

1. Purpose: Perform background tasks like garbage collection, monitoring, or housekeeping.
2. JVM Behavior: The JVM terminates automatically when only daemon threads are running.
3. Default Threads: Most threads you create are user threads; daemon

Concurrency Utilities

31. CountdownLatch, CyclicBarrier, and Semaphore

Concept	Definition	Use Case / Behavior
CountDownLatch	A synchronization aid that allows one or more threads to wait until a set of operations complete.	Count is decremented using <code>countDown()</code> . Waiting threads block on <code>await()</code> . One-time use.
CyclicBarrier	A barrier that allows a set of threads to wait for each other to reach a common point.	Reusable after all threads reach the barrier. Often used in parallel algorithms.

Concept	Definition	Use Case / Behavior
Semaphore	A permit-based synchronization tool controlling access to a resource.	acquire() reduces permits, release() increases. Useful for limiting concurrent access.

32. What is Exchanger in Java concurrency?

Definition:

Exchanger<V> is a synchronization point where two threads can exchange data objects. Each thread presents an object and receives the object from its partner thread.

Use Case:

- Thread coordination with data swapping.

33. What is Phaser in Java 7?

Phaser is a flexible reusable synchronization barrier, like `CyclicBarrier` or `CountDownLatch`, supporting multiple phases of tasks.

- Threads register and arrive at phases.
- Supports dynamic addition/removal of parties.
- Phaser is a synchronization barrier introduced in Java 7 (`java.util.concurrent`) that allows multiple threads to coordinate execution in phases.
- It is a more flexible and reusable alternative to:
 - `CountDownLatch` (one-time use)
 - `CyclicBarrier` (fixed number of threads)
- Key Concept
 - Threads register with the Phaser
 - They perform work
 - Then call `arriveAndAwaitAdvance()`
 - All threads wait until everyone reaches the same phase
 - Then move to the next phase
- Important Methods

• Method	• Description
• <code>register()</code>	• Register a new party (thread)
• <code>arrive()</code>	• Signal arrival without waiting

- Method
- arriveAndAwaitAdvance()
- arriveAndDeregister()
- getPhase()
- Description
- Signal arrival and wait
- Arrive and remove thread
- Get current phase number

- Example

- `import java.util.concurrent.Phaser;`

```
class PhaserExample {
    public static void main(String[] args) {

        Phaser phaser = new Phaser(3); // 3 threads

        Runnable task = () -> {
            System.out.println(Thread.currentThread().getName() + " Phase 1");
            phaser.arriveAndAwaitAdvance();

            System.out.println(Thread.currentThread().getName() + " Phase 2");
            phaser.arriveAndAwaitAdvance();

            System.out.println(Thread.currentThread().getName() + " Phase 3");
            phaser.arriveAndDeregister();
        };

        new Thread(task).start();
        new Thread(task).start();
        new Thread(task).start();
    }
}
```

- Output Flow
- All threads complete Phase 1 → move to Phase 2
All threads complete Phase 2 → move to Phase 3
- Why Use Phaser?
- Advantages
- Supports multiple phases (reusable)
- Allows dynamic registration/deregistration
- More flexible than CyclicBarrier
- Use Cases
- Multi-step processing (e.g., pipelines)
- Parallel algorithms with stages

- Game development (levels/phases sync)
 - Batch processing
 - Comparison
- | | | | |
|-------------------|------------------|-----------------|------------|
| • Feature | • CountdownLatch | • CyclicBarrier | • Phaser |
| • Reusable | • No | • Yes | • Yes |
| • Dynamic Threads | • No | • No | • Yes |
| • Phases | • No | • Limited | • Multiple |

Important Points

- If a thread fails to arrive → others may wait indefinitely
- Slightly complex compared to other synchronizers
- Best when phases are clearly defined

Phaser is a flexible synchronization barrier that allows multiple threads to coordinate execution across multiple phases with dynamic registration.

34. Explain ConcurrentHashMap.

ConcurrentHashMap is a thread-safe implementation of Map from java.util.concurrent that allows concurrent read and write operations without locking the entire map.

It is designed for high concurrency and performance.

How it Works (Core Idea)

Java 7 (Old Concept)

- Used Segment-based locking
- Map divided into segments → only a segment locked at a time

Java 8+ (Modern Approach)

- Uses:
 - CAS (Compare-And-Swap)
 - Synchronized blocks on buckets (only when needed)
- Locks only a small part (bucket/node) instead of whole map

Result: Better scalability and performance

Key Features

- Thread-safe without full synchronization
- High performance in concurrent environments

- Allows multiple threads to read and write simultaneously
- No ConcurrentModificationException during iteration
- Does not allow null keys or values

Example

```
import java.util.concurrent.ConcurrentHashMap;

class Example {
    public static void main(String[] args) {
        ConcurrentHashMap<String, Integer> map = new ConcurrentHashMap<>();

        map.put("A", 1);
        map.put("B", 2);

        map.forEach((k, v) -> {
            map.put("C", 3); // Safe during iteration
            System.out.println(k + " = " + v);
        });
    }
}
```

Important Methods

- put(key, value) → thread-safe insert/update
- get(key) → non-blocking read
- putIfAbsent(key, value) → atomic check-then-act
- compute(), computeIfAbsent() → atomic operations
- remove(key) → thread-safe removal

35. Difference: HashMap vs ConcurrentHashMap

Feature	HashMap	ConcurrentHashMap
Thread Safety	Not thread-safe	Thread-safe
Null Key/Value	Allows null key & value	Does not allow null key or value
Synchronization	Manual synchronization needed	Internal fine-grained locking / CAS
Performance	Faster in single-thread	Faster under concurrent access

36. Explain BlockingQueue and its types.

BlockingQueue is a thread-safe queue supporting blocking operations: waiting for the queue to become non-empty when retrieving or non-full when adding.

Types:

- ArrayBlockingQueue – fixed size array-based
- LinkedBlockingQueue – optionally bounded, linked list
- PriorityBlockingQueue – priority-based ordering
- SynchronousQueue – each insert waits for a remove

37. What is ReadWriteLock? When should it be used?

ReadWriteLock is an interface from `java.util.concurrent.locks` that allows separate locks for reading and writing.

It provides:

- Read Lock (`readLock()`) → multiple threads can read simultaneously
- Write Lock (`writeLock()`) → only one thread can write at a time

The most common implementation is `ReentrantReadWriteLock`.

How it Works

- Multiple threads can acquire read lock together (if no writer is active)
- Only one thread can acquire write lock (exclusive access)
- If a writer acquires the lock → all readers and other writers are blocked

Example

```
import java.util.concurrent.locks.ReadWriteLock;
import java.util.concurrent.locks.ReentrantReadWriteLock;
```

```
class SharedData {
    private int value = 0;
    private final ReadWriteLock lock = new ReentrantReadWriteLock();

    public int read() {
        lock.readLock().lock();
        try {
            return value;
        } finally {
            lock.readLock().unlock();
        }
    }

    public void write(int newValue) {
        lock.writeLock().lock();
        try {
            value = newValue;
        }
    }
}
```

```
    } finally {  
        lock.writeLock().unlock();  
    }  
}  
}
```

Why Use ReadWriteLock?

Compared to synchronized or a simple lock:

- Improves performance in read-heavy scenarios
- Allows higher concurrency (multiple readers at once)

When Should You Use It?

Use ReadWriteLock when:

- You have many read operations and few write operations
- Reads are independent and don't modify data
- Data is shared across threads

Real-World Scenarios

- Caching systems
- Configuration data (rarely updated, frequently read)
- In-memory data stores
- Shared collections (like maps)

When NOT to Use

- When writes are frequent (no benefit over normal lock)
- When logic is simple (adds unnecessary complexity)
- When contention is low

Important Points

- Writers get exclusive access
- Can lead to writer starvation if many readers keep coming
- Slightly more complex than ReentrantLock

ReadWriteLock allows multiple concurrent readers and a single writer, improving performance in read-heavy concurrent applications

38. Difference between ReentrantLock and ReentrantReadWriteLock.

ReentrantLock is a mutual exclusion (exclusive) lock where only one thread can access the critical section at a time.

Key Features

- Only one thread can hold the lock
- Supports reentrancy (same thread can acquire multiple times)
- Provides advanced APIs:
 - lock()
 - tryLock()
 - lockInterruptibly()
- Optional fairness policy

Example

```
ReentrantLock lock = new ReentrantLock();
```

```
lock.lock();  
try {  
    // critical section  
} finally {  
    lock.unlock();  
}
```

Use Case

- When write operations are frequent
- When you need fine control over locking

2. ReentrantReadWriteLock

Definition

ReentrantReadWriteLock provides two separate locks:

- Read Lock → multiple threads can read simultaneously
- Write Lock → only one thread can write (exclusive)

Key Features

- Read-Write separation
- Multiple readers allowed concurrently
- Writers get exclusive access
- Better performance for read-heavy scenarios

Example

```
ReentrantReadWriteLock rwLock = new ReentrantReadWriteLock();
```

```
rwLock.readLock().lock();  
try {  
    // read operation  
} finally {  
    rwLock.readLock().unlock();  
}
```

```
rwLock.writeLock().lock();  
try {  
    // write operation  
} finally {  
    rwLock.writeLock().unlock();  
}
```

Use Case

- When reads are frequent and writes are rare
- Example: caching, shared configuration

Key Differences

Feature	ReentrantLock	ReentrantReadWriteLock
Lock Type	Exclusive	Shared (read) + Exclusive (write)
Concurrent Reads	Not allowed	Allowed
Concurrent Writes	Not allowed	Not allowed
Performance	Lower in read-heavy cases	Better in read-heavy cases
Complexity	Simple	More complex
Use Case	Write-heavy	Read-heavy

Important Concept

In ReentrantReadWriteLock:

- Multiple readers can access only if no writer holds the lock
- If a writer acquires the lock → all readers are blocked

Pitfalls

- ReentrantReadWriteLock can lead to writer starvation (if many readers keep coming)
- Slightly more complex to implement correctly

ReentrantLock provides exclusive locking, while ReentrantReadWriteLock improves concurrency by allowing multiple readers and a single writer, making it ideal for read-heavy workloads.

39. What is ThreadLocal? Give an example.

ThreadLocal is a class in java.lang that provides thread-local variables — meaning each thread has its own independent copy of a variable.

Even if multiple threads access the same ThreadLocal object, they cannot see each other's values.

Why use ThreadLocal?

- To avoid shared state issues
- To achieve thread safety without synchronization
- Useful when each thread needs its own isolated data

How it Works

- Internally, each thread maintains a ThreadLocalMap
- When you call:
 - set() → value is stored for the current thread
 - get() → retrieves value for the current thread only

Example

```
class ThreadLocalExample {  
  
    private static ThreadLocal<Integer> threadLocal = ThreadLocal.withInitial(() -> 0);  
  
    public static void main(String[] args) {  
  
        Runnable task = () -> {  
            int value = threadLocal.get();  
            value++;  
            threadLocal.set(value);  
  
            System.out.println(Thread.currentThread().getName() + " Value: " + threadLocal.get());  
        };  
  
        Thread t1 = new Thread(task);  
        Thread t2 = new Thread(task);  
  
        t1.start();  
        t2.start();  
    }  
}
```

Output (Example)

Thread-0 Value: 1
Thread-1 Value: 1

Each thread gets its own copy, so both print 1 (not shared).

Real-World Use Cases

- Database connections per thread
- User session handling
- Date formatting (SimpleDateFormat is not thread-safe)
- Transaction management in frameworks like Spring

Important Methods

- `set(value)` → Set value for current thread
- `get()` → Get value for current thread
- `remove()` → Remove value (important to avoid memory leaks)

Common Pitfall

Memory Leak Risk

If you don't call `remove()` in thread pools, values may persist and cause memory issues.

ThreadLocal provides thread-specific storage, allowing each thread to have its own independent copy of a variable, avoiding synchronization issues.

40. Explain CopyOnWriteArrayList and CopyOnWriteArraySet.

`CopyOnWriteArrayList` is a thread-safe variant of `ArrayList` from `java.util.concurrent` where all write operations (add, remove, update) create a new copy of the underlying array.

how it Works

- On read operations → no locking (fast, concurrent access allowed)
- On write operations → a new copy of the array is created, modified, and replaced

This ensures no `ConcurrentModificationException` during iteration.

Key Characteristics

- Thread-safe without explicit synchronization
- Iterators are fail-safe (work on snapshot, not original data)
- Best for read-heavy, write-light scenarios

Example

```
CopyOnWriteArrayList<String> list = new CopyOnWriteArrayList<>();
```

```
list.add("A");
```

```
list.add("B");
```

```
for (String item : list) {
```

```
list.add("C"); // No ConcurrentModificationException
}
```

```
System.out.println(list);
```

Use Cases

- Event listener lists
- Caching data
- Read-heavy applications (like configurations)

Drawbacks

- High memory usage (copy on every write)
- Slow write performance

2. CopyOnWriteArraySet

Definition

CopyOnWriteArraySet is a thread-safe Set implementation backed by a CopyOnWriteArrayList.

How it Works

- Internally uses CopyOnWriteArrayList
- Ensures no duplicate elements (Set property)
- Same copy-on-write mechanism for thread safety

Key Characteristics

- Thread-safe
- No duplicates allowed
- Iterators are fail-safe
- Maintains insertion order

Example

```
CopyOnWriteArraySet<String> set = new CopyOnWriteArraySet<>();
```

```
set.add("A");
set.add("B");
set.add("A"); // Duplicate ignored
```

```
for (String item : set) {
    set.add("C"); // Safe during iteration
}
```

```
System.out.println(set);
```

Use Cases

- Maintaining unique listeners/subscribers
- Read-heavy concurrent Set operations

Drawbacks

- Same as CopyOnWriteArrayList:
 - Expensive writes
 - Memory overhead

Key Differences

Feature	CopyOnWriteArrayList	CopyOnWriteArraySet
Type	List	Set
Duplicates	Allowed	Not allowed
Backing Structure	Array	CopyOnWriteArrayList
Order	Maintains insertion order	Maintains insertion order
Thread Safety	Yes	Yes

CopyOnWriteArrayList and CopyOnWriteArraySet are thread-safe collections that achieve concurrency by copying the entire underlying array on write operations, making them ideal for read-heavy scenarios.

41. How to handle thread interruption?

Thread interruption is a mechanism to signal a thread that it should stop what it's doing. The thread checks for the interruption signal and terminates gracefully instead of being forcefully stopped.

Key Point: You never force a thread to stop; you politely ask it to stop using `interrupt()`.

2. How It Works

1. Any thread can call `thread.interrupt()` to set the interrupt flag.
2. The running thread should periodically check the flag using:

`Thread.currentThread().isInterrupted()`

3. If the thread is blocked (e.g., `sleep()`, `wait()`, `join()`), it throws `InterruptedException`.

Example

```
public class InterruptExample implements Runnable {
    @Override
```

```

public void run() {
    while (!Thread.currentThread().isInterrupted()) { // check interrupt flag
        try {
            System.out.println("Thread running...");
            Thread.sleep(1000); // may throw InterruptedException
        } catch (InterruptedException e) {
            System.out.println("Thread interrupted, stopping...");
            Thread.currentThread().interrupt(); // preserve interrupt status
            break;
        }
    }
}

public static void main(String[] args) throws InterruptedException {
    Thread t = new Thread(new InterruptExample());
    t.start();
    Thread.sleep(3000);
    t.interrupt(); // signal thread to stop
}
}

```

Explanation:

- `interrupt()` signals the thread to stop.
- `isInterrupted()` is used to check the flag.
- Catching `InterruptedException` is necessary for blocking operations, and you should restore the interrupt flag.

Interview Answer

“Thread interruption in Java is a way to gracefully stop a thread.

One thread calls `interrupt()` on another thread to signal it, and the target thread checks the interrupt flag or handles `InterruptedException` if it’s blocked.

This approach ensures threads can stop safely without abrupt termination.”

42. How to detect deadlock in Java?

Concept:

A deadlock occurs when two or more threads are blocked forever, each waiting for a lock held by the other.

Example scenario with two threads and two resources:

Thread 1: holds Lock A → waits for Lock B

Thread 2: holds Lock B → waits for Lock A

Deadlock occurs when threads wait indefinitely for each other’s locks.

Detection Methods:

1. Thread dump analysis (jstack) – check for circular waits.
2. Java Management API: ThreadMXBean.findDeadlockedThreads().
3. Logging lock acquisition order – detect circular dependencies.

Scenario:

- Use when multiple locks are acquired in different orders by multiple threads.

Deadlocks can occur when multiple threads hold locks and wait for others. In Java, I can detect deadlocks using ThreadMXBean.findDeadlockedThreads() or thread dumps. When using wait() and notify(), I avoid deadlocks by acquiring locks in a consistent order and minimizing nested synchronized blocks.

43. How do you implement a producer-consumer problem in Java?

The Producer-Consumer problem is a classic concurrency problem in which one or more producer threads generate data and place it into a shared buffer, while one or more consumer threads remove and process the data.

The challenge is to synchronize access to the shared resource so that:

1. Producers wait if the buffer is full.
2. Consumers wait if the buffer is empty.

The goal is to avoid race conditions, data loss, and thread interference while maintaining efficient processing.

In Java, it can be implemented using wait()/notify() with a synchronized block or more efficiently using BlockingQueue, which handles thread safety internally

44. How do you implement a thread-safe singleton?

A Singleton is a design pattern that:

1. Ensures only one instance of a class exists.
2. Provides a global point of access to that instance.

Using Bill Pugh Singleton (Best Practice):

```
class Singleton {  
    private Singleton() {}  
    private static class Holder {  
        private static final Singleton INSTANCE = new Singleton();  
    }  
    public static Singleton getInstance() {  
        return Holder.INSTANCE;  
    }  
}
```

Scenario:

- Use in multi-threaded caching, configuration loading, or logging utilities.
- In modern Java, Bill Pugh Singleton or Enum Singleton are usually the go-to answers for interviews because they're concise, lazy, and thread-safe without complex synchronization.

45. How do you prevent concurrent modification exceptions?

A `ConcurrentModificationException` occurs when you modify a collection while iterating over it using methods that are not thread-safe or fail-fast.

Example:

```
import java.util.ArrayList;
import java.util.List;

public class CMExample {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>();
        list.add("A");
        list.add("B");
        list.add("C");

        for (String s : list) {
            if (s.equals("B")) {
                list.remove(s); // ❌ Causes ConcurrentModificationException
            }
        }
    }
}
```

Why:

- The iterator detects that the underlying collection was structurally modified outside of its own methods.

2. How to Prevent It

a) Use Iterator's `remove()` method

The safe way to remove elements during iteration:

```
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

public class SafeIterator {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>(List.of("A", "B", "C"));
    }
}
```

```

        Iterator<String> iterator = list.iterator();
        while (iterator.hasNext()) {
            String s = iterator.next();
            if (s.equals("B")) {
                iterator.remove(); // Safe
            }
        }

        System.out.println(list); // [A, C]
    }
}

```

Key: Always use iterator's remove, not list.remove().

b) Use CopyOnWriteArrayList for Multi-threading

If multiple threads read/write a list concurrently:

```

import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;

public class CopyOnWriteExample {
    public static void main(String[] args) {
        List<String> list = new CopyOnWriteArrayList<>();
        list.add("A");
        list.add("B");
        list.add("C");

        for (String s : list) {
            if (s.equals("B")) {
                list.remove(s); // Safe, no CME
            }
        }

        System.out.println(list); // [A, C]
    }
}

```

Key:

- CopyOnWriteArrayList creates a snapshot for iteration.
- Expensive for frequent writes but safe for concurrent reads + occasional modifications.

c) Collect and Remove Later

If you don't want iterators or special lists:

```

import java.util.ArrayList;
import java.util.List;

public class CollectAndRemove {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>(List.of("A", "B", "C"));
        List<String> toRemove = new ArrayList<>();

        for (String s : list) {
            if (s.equals("B")) {
                toRemove.add(s);
            }
        }

        list.removeAll(toRemove); // Safe
        System.out.println(list); // [A, C]
    }
}

```

d) Use synchronized Collections

For multi-threaded access:

```

import java.util.Collections;
import java.util.List;
import java.util.ArrayList;

public class SynchronizedExample {
    public static void main(String[] args) {
        List<String> list = Collections.synchronizedList(new ArrayList<>());
        list.add("A");
        list.add("B");
        list.add("C");

        synchronized (list) {
            for (String s : list) {
                if (s.equals("B")) {
                    list.remove(s); // Safe inside synchronized block
                }
            }
        }
    }
}

```

In high-concurrency scenarios, I prefer `CopyOnWriteArrayList` or concurrent maps/sets like `ConcurrentHashMap` to avoid CMEs, as they provide thread-safe iteration without locking the whole collection

46. How to limit the number of concurrent threads in an application?

Limiting the number of concurrent threads in an application is essential to prevent resource exhaustion and maintain predictable performance. In Java, there are multiple ways to achieve this depending on the use case. Let's break it down carefully.

1. Using ExecutorService with a Fixed Thread Pool

Java's ExecutorService provides thread pools to manage concurrency. A fixed thread pool restricts the number of active threads.

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class ThreadLimitExample {
    public static void main(String[] args) {
        int maxThreads = 5; // Limit to 5 concurrent threads
        ExecutorService executor = Executors.newFixedThreadPool(maxThreads);

        for (int i = 0; i < 10; i++) {
            int taskId = i;
            executor.submit(() -> {
                System.out.println("Running task " + taskId + " by " +
Thread.currentThread().getName());
                try {
                    Thread.sleep(2000); // Simulate work
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            });
        }

        executor.shutdown();
    }
}
```

Explanation:

- Only 5 threads run concurrently.
 - Additional tasks wait in a queue until a thread becomes available.
 - This approach avoids creating too many threads and overwhelming the system.
-

2. Using Semaphore to Control Access

A Semaphore can be used to limit concurrent access to a resource.

```
import java.util.concurrent.Semaphore;
```

```

public class SemaphoreExample {
    public static void main(String[] args) {
        int maxThreads = 3;
        Semaphore semaphore = new Semaphore(maxThreads);

        for (int i = 0; i < 10; i++) {
            int taskId = i;
            new Thread(() -> {
                try {
                    semaphore.acquire(); // Acquire permit
                    System.out.println("Task " + taskId + " is running");
                    Thread.sleep(2000); // Simulate work
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                } finally {
                    semaphore.release(); // Release permit
                }
            }).start();
        }
    }
}

```

Explanation:

- Only 3 threads can execute simultaneously.
- Other threads block until a permit is available.
- Useful when you want fine-grained control without a thread pool.

3. Using BlockingQueue for Producer-Consumer Patterns

You can use a bounded BlockingQueue to control how many tasks are in progress:

```

import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.BlockingQueue;

public class BlockingQueueExample {
    public static void main(String[] args) {
        int maxThreads = 4;
        BlockingQueue<Runnable> queue = new ArrayBlockingQueue<>(maxThreads);

        for (int i = 0; i < 10; i++) {
            int taskId = i;
            queue.add(() -> {
                System.out.println("Executing task " + taskId);
            });
        }
    }
}

```



```
        queue.forEach(Runnable::run); // Simple execution example
    }
}
```

Explanation:

- The queue size limits how many tasks can wait.
 - Only a fixed number of threads should be created to poll and process the queue.
-

4. Other Considerations

- ForkJoinPool: For parallel streams or divide-and-conquer tasks, you can set parallelism level.
- ThreadPoolExecutor: More flexible than Executors.newFixedThreadPool(); allows queue size, keep-alive time, and rejection policies.
- Avoid creating too many threads manually, as it can cause CPU thrashing and OutOfMemoryError.
- Always shutdown your executors properly to avoid resource leaks.

Best Practice:

For most cases, using a fixed-size thread pool via ExecutorService is the cleanest and safest way to limit concurrent threads in Java.

47. How do you prioritize threads in Java?

Java threads have priority levels that hint to the thread scheduler which threads are more important.

Thread priority affects the order in which threads are scheduled, but it does not guarantee execution order.

Thread Priority Levels

- Thread.MIN_PRIORITY = 1
- Thread.NORM_PRIORITY = 5 (default)
- Thread.MAX_PRIORITY = 10

How to Set Thread Priority

1. Using setPriority() method

```
Thread t1 = new Thread(() -> {
    System.out.println("Thread 1 running");
});
```

```
Thread t2 = new Thread(() -> {
    System.out.println("Thread 2 running");
});
```

```
});
```

```
t1.setPriority(Thread.MAX_PRIORITY); // 10  
t2.setPriority(Thread.MIN_PRIORITY); // 1
```

```
t1.start();  
t2.start();
```

2. Using `getPriority()` to check

```
System.out.println(t1.getPriority()); // 10  
System.out.println(t2.getPriority()); // 1
```

1. Thread priority is only a hint; actual scheduling depends on the JVM and OS thread scheduler.
2. Higher-priority threads are more likely to be executed before lower-priority threads.
3. Avoid relying on priority for program correctness; use synchronization or `ExecutorService` for deterministic behavior.

48. Difference between `join()` and `wait()`

`join()`

`join()` is a method of the `Thread` class that allows one thread to wait for another thread to finish execution.

The calling thread pauses until the target thread completes or a specified timeout expires.

`wait()`

`wait()` is a method of the `Object` class that allows a thread to release the lock it holds and wait until another thread notifies it (`notify()` or `notifyAll()`).

It is used for inter-thread communication within synchronized blocks.

Key Differences

Feature	<code>join()</code>	<code>wait()</code>
Belongs To	<code>Thread</code> class	<code>Object</code> class
Purpose	Wait for a thread to finish	Wait for a condition or notification
Synchronization	No need to hold a lock	Must be called inside synchronized block
Resumed By	Thread finishes execution	Another thread calls <code>notify()</code> / <code>notifyAll()</code>
Blocking Behavior	Blocks calling thread	Blocks calling thread and releases lock

Example

Using `join()`

```
Thread t1 = new Thread(() -> {  
    System.out.println("Thread t1 running");
```

```
});
t1.start();
t1.join(); // main thread waits for t1
System.out.println("t1 has finished");
```

Using wait()

```
Object lock = new Object();
Thread t1 = new Thread(() -> {
    synchronized(lock) {
        try { lock.wait(); } catch (InterruptedException e) {}
        System.out.println("Thread resumed after notify");
    }
});
```

```
t1.start();
```

```
synchronized(lock) {
    lock.notify(); // wakes up t1
}
```

Interview Tip

Use join() when you want to wait for thread completion, and wait() when threads need to communicate or coordinate via shared resources.

49. What are best practices for designing multithreaded applications?

Designing multithreaded applications requires careful attention to thread safety, resource management, and performance, while minimizing risks like race conditions, deadlocks, and contention.

Best Practices

1. Use High-Level Concurrency APIs
 - Prefer ExecutorService, ForkJoinPool, BlockingQueue, ConcurrentHashMap over manual Thread and synchronized.
 - Benefits: simpler code, better performance, easier maintenance.
2. Minimize Synchronization Scope
 - Lock only critical sections, not entire methods.
 - Use fine-grained locks or ReadWriteLock where applicable.
3. Prefer Immutable Objects
 - Immutable objects are inherently thread-safe.
 - Reduces the need for locking and prevents data corruption.
4. Avoid Deadlocks
 - Acquire multiple locks in a consistent order.

- Use `tryLock()` with timeout for safer lock acquisition.
5. Handle Thread Interruption Gracefully
 - Check `Thread.currentThread().isInterrupted()` periodically.
 - Use interruptible blocking operations like `BlockingQueue.take()`.
 6. Use Thread-Safe Collections
 - Use `ConcurrentHashMap`, `CopyOnWriteArrayList`, `BlockingQueue` instead of manually synchronized collections.
 7. Limit Thread Creation
 - Use thread pools (`Executors.newFixedThreadPool`) instead of creating unbounded threads.
 - Helps control CPU and memory usage.
 8. Separate Read and Write Operations When Possible
 - Use `ReentrantReadWriteLock` for read-heavy workloads, allowing multiple concurrent readers.
 9. Provide Proper Exception Handling
 - Catch exceptions inside threads to prevent silent thread termination.
 - Use `Thread.setUncaughtExceptionHandler` for logging unhandled exceptions.
 10. Monitor and Debug Threads
 - Include thread names in logs.
 - Use profilers, thread dumps, and metrics to identify bottlenecks or deadlocks.
 11. Test Under Concurrency
 - Simulate multi-threaded scenarios in unit tests.
 - Stress-test for race conditions, deadlocks, and data consistency.

Summary for Interviews

Focus on thread safety, minimizing contention, using high-level concurrency utilities, handling exceptions, and testing under concurrent load.

50. How do you debug multithreaded applications effectively?

Debugging multithreaded applications is challenging due to concurrent execution, race conditions, deadlocks, and thread interleaving. Effective debugging requires thread-aware tools, logging, and systematic analysis.

Strategies / Best Practices

1. Thread Dumps / Stack Traces
 - Use `jstack` or IDE thread analyzers to capture the state of all threads.

- Identify blocked threads, waiting threads, and deadlocks.

2. Logging with Thread Information

- Include thread name and ID in log statements.
- Example:

```
System.out.println(Thread.currentThread().getName() + ": Processing task");
```

- Helps trace thread execution flow and detect race conditions.

3. Deadlock Detection

- Use Java Management API (ThreadMXBean):

```
ThreadMXBean bean = ManagementFactory.getThreadMXBean();  
long[] deadlocked = bean.findDeadlockedThreads();
```

- VisualVM or profilers can also detect deadlocks.

4. Use Thread-Safe Collections & High-Level Concurrency APIs

- Reduces issues from manual synchronization.
- Example: ConcurrentHashMap, CopyOnWriteArrayList, BlockingQueue.

5. Conditional Breakpoints / Debugging Tools

- Use IDEs (IntelliJ, Eclipse) with thread breakpoints or suspend/resume specific threads.
- Avoid suspending all threads unless necessary.

6. Unit Testing and Stress Tests

- Simulate concurrent scenarios with multithreaded JUnit tests.
- Helps catch race conditions and intermittent bugs early.

7. Profilers & Monitoring Tools

- Use VisualVM, YourKit, or JProfiler to monitor CPU usage, thread states, and memory usage.
- Detect threads stuck in loops, deadlocks, or excessive blocking.

8. Reproduce with Deterministic Scenarios

- Use Thread.sleep, CountdownLatch, or CyclicBarrier to control thread execution order for testing.

